

i2c-pseudo driver

Usually I2C adapters are implemented in a kernel driver. It is also possible to implement an adapter in userspace, through the `/dev/i2c-pseudo-controller` interface. Load module `i2c-pseudo` for this.

Use cases for this module include:

- Using local I2C device drivers, particularly `i2c-dev`, with I2C busses on remote systems. For example, interacting with a Device Under Test (DUT) connected to a Linux host through a debug interface, or interacting with a remote host over a network.
- Implementing I2C device driver tests that are impractical with the `i2c-stub` module. For example, when simulating an I2C device where its driver might issue a sequence of reads and writes without interruption, and the value at a certain address must change during the sequence.

This is not intended to replace kernel drivers for actual I2C busses on the local host machine.

Details

Each time `/dev/i2c-pseudo-controller` is opened, and the correct initialization command is written to it (`ADAPTER_START`), a new I2C adapter is created. The adapter will live until its file descriptor is closed. Multiple pseudo adapters can co-exist simultaneously, controlled by the same or different userspace processes. When an I2C device driver sends an I2C message to a pseudo adapter, the message becomes readable from its file descriptor. If a reply is written before the adapter timeout expires, that reply will be sent back to the I2C device driver.

Reads and writes are buffered inside `i2c-pseudo` such that userspace controllers may split them up into arbitrarily small chunks. Multiple commands, or portions of multiple commands, may be read or written together.

Blocking I/O is the default. Non-blocking I/O is supported as well, enabled by `O_NONBLOCK`. Polling is supported, with or without non-blocking I/O. A special command (`ADAPTER_SHUTDOWN`) is available to unblock any pollers or blocked reads or writes, as a convenience for a multi-threaded or multi-process program that wants to exit.

It is safe to access a single controller fd from multiple threads or processes concurrently, though it is up to the controller to ensure proper ordering, and to ensure that writes for different commands do not get interleaved. However, it is recommended (not required) that controller implementations have only one reader thread and one writer thread, which may or may not be the same thread. Avoiding multiple readers and multiple writers greatly simplifies controller implementation, and there is likely no performance benefit to be gained from concurrent reads or concurrent writes due to how i2c-pseudo serializes them internally. After all, on a real I2C bus only one I2C message can be active at a time.

Commands are newline-terminated, both those read from the controller device, and those written to it.

Read Commands

The commands that may be read from a pseudo controller device are:

Read Command I2C_ADAPTER_NUM <num>

Example "I2C_ADAPTER_NUM 5\n"

Details This is read in response to the GET_ADAPTER_NUM command being written. The number is the I2C adapter number in decimal. This can only occur after ADAPTER_START, because before that the number is not known and cannot be predicted reliably.

Read Command I2C_PSEUDO_ID <num>

Example "I2C_PSEUDO_ID 98\n"

Details This is read in response to the GET_PSEUDO_ID command being written. The number is the pseudo ID in decimal.

Read Command I2C_BEGIN_XFER

Example "I2C_BEGIN_XFER\n"

Details This indicates the start of an I2C transaction request, in other words the start of the I2C messages from a single invocation of the I2C adapter's master_xfer() callback. This can only occur after ADAPTER_START.

Read Command I2C_XFER_REQ <xfer_id> <msg_id> <addr> <flags>
<data_len> [<write_byte>[:...]]

Example "I2C_XFER_REQ 3 0 0x0070 0x0000 2 AB:9F\n"

Example "I2C_XFER_REQ 3 1 0x0070 0x0001 4\n"

Details This is a single I2C message that a device driver requested be sent on the bus, in other words a single struct `i2c_msg` from `master_xfer()` `msgs` arg.

The `xfer_id` is a number representing the whole I2C transaction, thus all I2C_XFER_REQ between a I2C_BEGIN_XFER + I2C_COMMIT_XFER pair share an `xfer_id`. The purpose is to ensure replies from the userspace controller are always properly matched to the intended `master_xfer()` request. The first transaction has `xfer_id` 0, and it increases by 1 with each transaction, however it will eventually wrap back to 0 if enough transactions happen during the lifetime of a pseudo adapter. It is guaranteed to have a large enough maximum value such that there can never be multiple outstanding transactions with the same ID, due to an internal limit in `i2c-pseudo` that will block `master_xfer()` calls when the controller is falling behind in its replies.

The `msg_id` is a decimal number representing the index of the I2C message within its transaction, in other words the index in `master_xfer()` `*msgs` array arg. This starts at 0 after each I2C_BEGIN_XFER. This is guaranteed to not wrap.

The `addr` is the hexadecimal I2C address for this I2C message. The address is right-aligned without any read/write bit.

The `flags` are the same bitmask flags used in struct `i2c_msg`, in hexadecimal form. Of particular importance to any pseudo controller is the read bit, which is guaranteed to be 0x1 per Linux I2C documentation.

The `data_len` is the decimal number of either how many bytes to write that will follow, or how many bytes to read and reply with if this is a read request.

If this is a read, `data_len` will be the final field in this command. If this is a write, `data_len` will be followed by the given number of colon-separated hexadecimal byte values, in the format shown in the example above.

Read Command I2C_COMMIT_XFER

Example "I2C_COMMIT_XFER\n"

Details This indicates the end of an I2C transaction request, in other words the end of the I2C messages from a single invocation of the I2C adapter's `master_xfer()` callback. This should be read exactly once after each

I2C_BEGIN_XFER, with a varying number of I2C_XFER_REQ between them.

Write Commands

The commands that may be written to a pseudo controller device are:

Write Command SET_ADAPTER_NAME_SUFFIX <suffix>

Example "SET_ADAPTER_NAME_SUFFIX My Adapter\n"

Details Sets a suffix to append to the auto-generated I2C adapter name. Only valid before ADAPTER_START. A space or other separator character will be placed between the auto-generated name and the suffix, so there is no need to include a leading separator in the suffix. If the resulting name is too long for the I2C adapter name field, it will be quietly truncated.

Write Command SET_ADAPTER_TIMEOUT_MS <ms>

Example "SET_ADAPTER_TIMEOUT_MS 2000\n"

Details Sets the timeout in milliseconds for each I2C transaction, in other words for each master_xfer() reply. Only valid before ADAPTER_START. The I2C subsystem will automatically time out transactions based on this setting. Set to 0 to use the I2C subsystem default timeout. The default timeout for new pseudo adapters where this command has not been used is configurable at i2c-pseudo module load time, and itself has a default independent from the I2C subsystem default. (If the i2c-pseudo module level default is set to 0, that has the same meaning as here.)

Write Command ADAPTER_START

Example "ADAPTER_START\n"

Details Tells i2c-pseudo to actually create the I2C adapter. Only valid once per open controller fd.

Write Command GET_ADAPTER_NUM

Example "GET_ADAPTER_NUM\n"

Details Asks i2c-pseudo for the number assigned to this I2C adapter by the I2C subsystem. Only valid after ADAPTER_START, because before that the number is not known and cannot be predicted reliably.

Write Command GET_PSEUDO_ID

Example "GET_PSEUDO_ID\n"

Details Asks i2c-pseudo for the pseudo ID of this I2C adapter. The pseudo ID will not be reused for the lifetime of the i2c-pseudo module, unless an internal counter wraps. I2C clients can use this to track specific instances of pseudo adapters, even when adapter numbers have been reused.

Write Command I2C_XFER_REPLY <xfer_id> <msg_id> <addr> <flags>
<errno> [<read_byte>[:...]]

Example "I2C_XFER_REPLY 3 0 0x0070 0x0000 0\n"

Example "I2C_XFER_REPLY 3 1 0x0070 0x0001 0 0B:29:02:D9\n"

Details This is how a pseudo controller can reply to I2C_XFER_REQ. Only valid after I2C_XFER_REQ. A pseudo controller should write one of these for each I2C_XFER_REQ it reads, including for failures, so that I2C device drivers need not wait for the adapter timeout upon failure (if failure is known sooner).

The fields in common with I2C_XFER_REQ have their same meanings, and their values are expected to exactly match what was read in the I2C_XFER_REQ command that this is in reply to.

The errno field is how the pseudo controller indicates success or failure for this I2C message. A 0 value indicates success. A non-zero value indicates a failure. Pseudo controllers are encouraged to use errno values to encode some meaning in a failure response, but that is not a requirement, and the I2C adapter interface does not provide a way to pass per-message errno values to a device driver anyways.

Pseudo controllers are encouraged to reply in the same order as messages were received, however i2c-pseudo will properly match up out-of-order replies with their original requests.

Write Command ADAPTER_SHUTDOWN

Example "ADAPTER_SHUTDOWN\n"

Details This tells i2c-pseudo that the pseudo controller wants to shutdown and intends to close the controller device fd soon. Use of this is OPTIONAL, it is perfectly valid to close the controller device fd without ever using this command.

This commands unblocks any blocked controller I/O (reads, writes, or

polls), and that is its main purpose.

Any I2C transactions attempted by a device driver after this command will fail, and will not be passed on to the userspace controller.

This DOES NOT delete the I2C adapter. Only closing the fd will do that. That MAY CHANGE in the future, such that this does delete the I2C adapter. (However this will never be required, it will always be okay to simply close the fd.)

Example userspace controller code

In C, a simple exchange between i2c-pseudo and userspace might look like the example below. Note that for brevity this lacks any error checking and handling, which a real pseudo controller implementation should have.

```
int fd;
char buf[1<<12];

fd = open("/dev/i2c-pseudo-controller", O_RDWR);
/* Create the I2C adapter. */
dprintf(fd, "ADAPTER_START\n");

/*
 * Pretend this I2C adapter number is 5, and the first I2C xfer sent to it was
 * from this command (using its i2c-dev interface):
 * $ i2cset -y 5 0x70 0xC2
 *
 * Then this read would place the following into *buf:
 * "I2C_BEGIN_XFER\n"
 * "I2C_XFER_REQ 0 0 0x0070 0x0000 1 C2\n"
 * "I2C_COMMIT_XFER\n"
 */
read(fd, buf, sizeof(buf));

/* This reply would allow the i2cset command above to exit successfully. */
dprintf(fd, "I2C_XFER_REPLY 0 0 0x0070 0x0000 0\n");

/*
 * Now pretend the next I2C xfer sent to this adapter was from:
 * $ i2cget -y 5 0x70 0xAB
 *
 * Then this read would place the following into *buf:
 * "I2C_BEGIN_XFER\n"
 * "I2C_XFER_REQ 1 0 0x0070 0x0000 1 AB\n"
 */
```

```

    * "I2C_XFER_REQ 1 1 0x0070 0x0001 1\n"
    * "I2C_COMMIT_XFER\n"
    */
    read(fd, buf, sizeof(buf));

    /*
     * These replies would allow the i2cget command above to print the following to
     * stdout and exit successfully:
     * 0x0b
     *
     * Note that it is also valid to write these together in one write().
     */
    dprintf(fd, "I2C_XFER_REPLY 1 0 0x0070 0x0000 0\n");
    dprintf(fd, "I2C_XFER_REPLY 1 1 0x0070 0x0001 0 0B\n");

    /* Destroy the I2C adapter. */
    close(fd);

```